EXPERIMENT 11: MAP COLORING USING CSP

Aim

To write a Python program to solve the **Map Coloring problem using Constraint Satisfaction Problem (CSP)**.

Objective

- To understand the concept of Constraint Satisfaction Problems.
- To represent map regions as variables.
- To assign colors from a predefined domain.
- To ensure that adjacent regions have different colors.
- To use backtracking to find a valid solution.

Algorithm

1. Define the regions of the map as variables.
2. Define the available colors as the domain.
3. Define the neighboring regions as constraints.
4. Select an uncolored region.
5. Assign a color to the selected region.
6. Check whether the color satisfies all constraints.
7. If valid, continue with the next region.
8. If no color is possible, backtrack and change the previous assignment.
9. Repeat until all regions are colored.
10. Display the solution.

Flowchart

```
     START
       |
       v
  Define Map & Colors
       |
       v
 Select Uncolored Region
       |
       v
```

```
  Select a Color
        |
        v
 Is Color Safe for
  All Neighbors?
      /     \
    YES      NO
     |        |
     v        v
 Assign Color  Try Next Color
     |
     v
 All Regions Colored?
      /     \
    YES      NO
     |        |
     v        |
 Display      |
 Solution <-----+
     |
     v
   END
```

Python Program

Writing

```
# Map Coloring using CSP

colors = ["Red", "Green", "Blue"]

graph = {
    "A": ["B", "C"],
```

```python
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}

color = {}

def is_safe(node, c):
    for neighbor in graph[node]:
        if neighbor in color and color[neighbor] == c:
            return False
    return True

def solve():
    if len(color) == len(graph):
        return True

    node = next(n for n in graph if n not in color)

    for c in colors:
        if is_safe(node, c):
            color[node] = c

            if solve():
                return True

            del color[node]

    return False
```

```
if solve():
    print("Map Coloring Solution:")
    for node in color:
        print(node, "->", color[node])
else:
    print("No solution")
```

Sample Output

Map Coloring Solution:

A -> Red

B -> Green

C -> Blue

D -> Red

Result

Thus, the **Map Coloring problem was successfully implemented using CSP and backtracking** in Python.